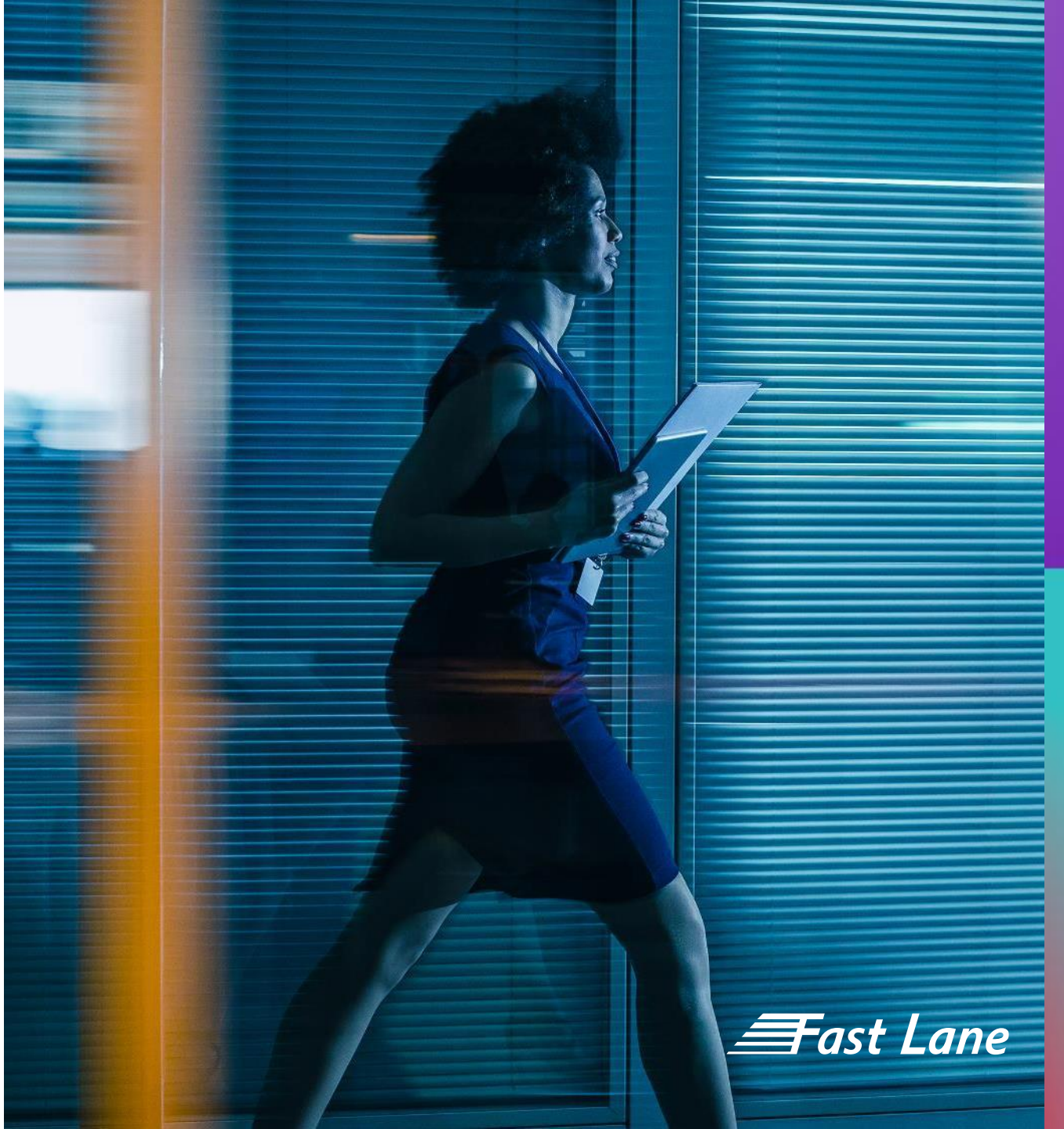
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 2

Configure the Harness

Rules, tools and guardrails around the model

Agentic Software Engineering with Claude Code



01

Explore the Harness Elements

What each harness file does and where it lives

02

Write the CLAUDE.md Constitution

CLAUDE.md, context files and output styles

03

Build Custom Slash Commands

Reusable workflows behind a slash

04

Automate Workflows with Hooks and Mods

Deterministic guardrails and harness extensions

05

Persist Corrections in Memory Files

Corrections that persist across sessions

The Claude Code harness

Rules, tools and guardrails around the model

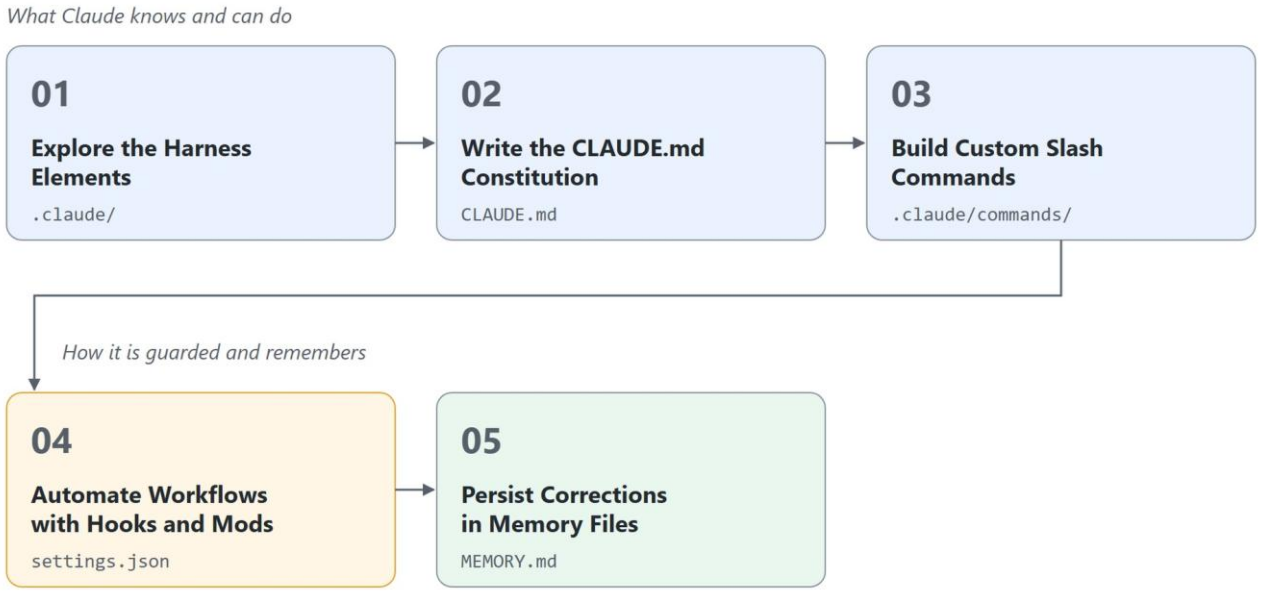
- The model is stateless; the **harness** makes it project-aware
- Files that decide what Claude knows, what it can do, and what it may do
- Tuned once, every turn in every session benefits
- Committed with the repo, so every teammate gets the same behavior



Overview

The path through the harness

- Start with the **harness elements**, the files around the model, and where they live
- Write the project constitution in **CLAUDE.md**
- Package repeated prompts as **slash commands**
- Guard with **hooks**, extend with mods
- Finish with **memory** that keeps corrections across sessions



01

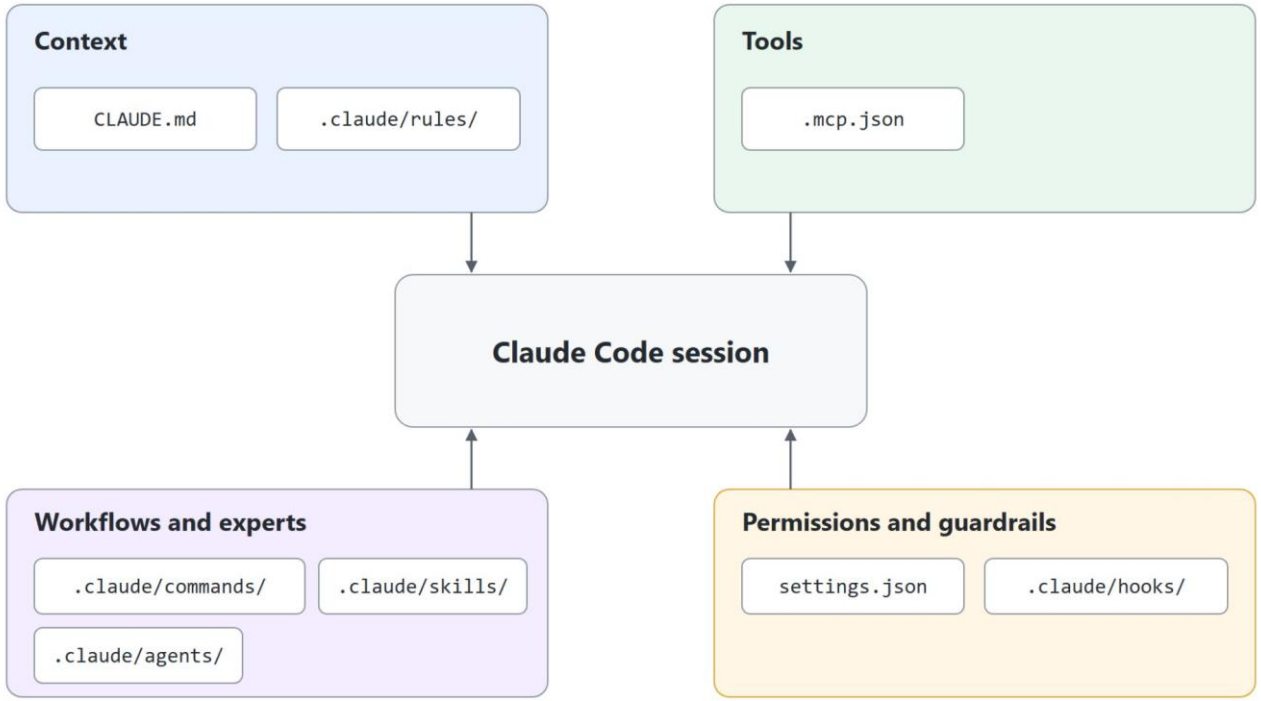
Explore the Harness Elements

What each harness file does and where it lives

Harness file types

One file per concern, loaded before your first prompt

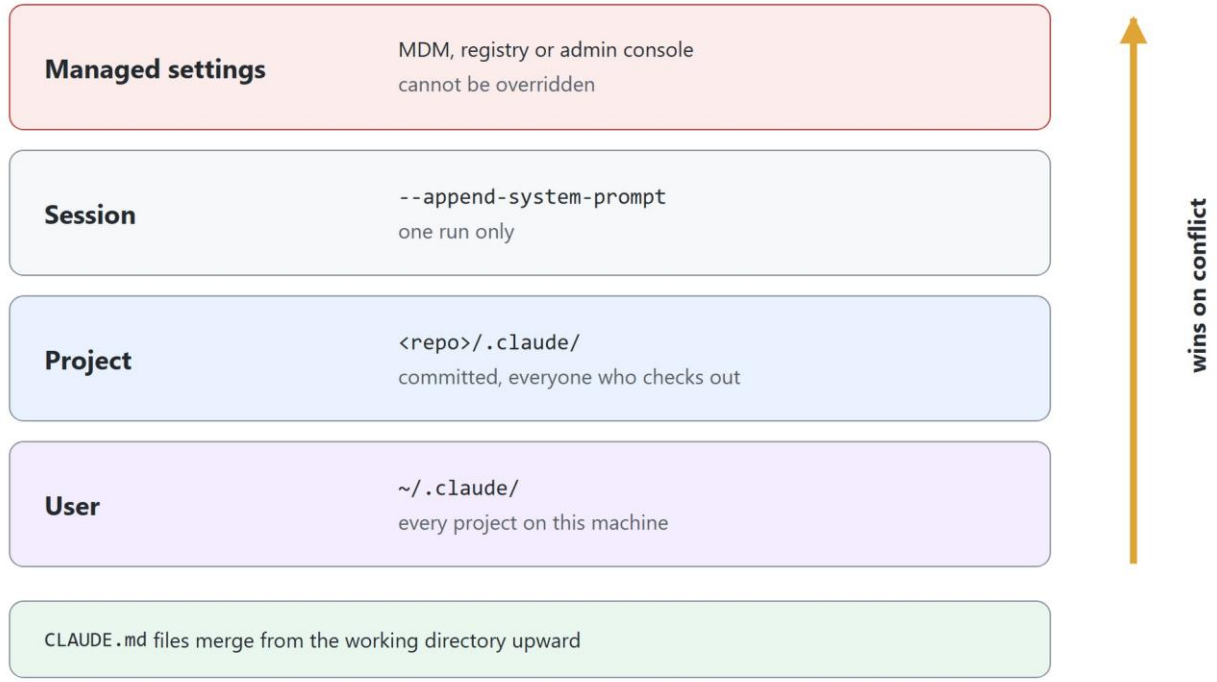
- **`CLAUDE.md`** and `.claude/rules/`: rules, architecture and standards
- **`settings.json`**: permissions, env vars, model and hook registrations
- **`.mcp.json`**: MCP servers that add external tools
- **`.claude/commands/`**, `skills/`, `agents/`: workflows, capabilities and experts
- **`.claude/hooks/`**: scripts that run on lifecycle events



Where harness files live

Managed, project, user and session scopes

- **User** `~/.claude/`: personal preferences and private credentials
- **Project** `.claude/`: committed, so every teammate inherits it
- **Session**: `--append-system-prompt` for a single run
- **Managed settings** sit above all three and cannot be overridden
- The project file wins when a setting appears in both user and project



02

Write the CLAUDE.md Constitution

CLAUDE.md, context files and output styles

CLAUDE.md and path-scoped rules

Context that loads every turn, rules that load on a match

- `CLAUDE.md` loads every turn in its subtree: keep it to always-true rules
- Global, project and nested files **merge** from the working directory upward
- `.claude/rules/` load at start; with `paths:` on the first matching read
- Two complete **examples** pin an output shape better than a paragraph
- `@path` attaches a file to the prompt so Claude need not search for it



Need	Mechanism
Whole project	Root `CLAUDE.md`
One subtree	Nested `CLAUDE.md`
Files matching a pattern	`.claude/rules/` with `paths:`
Facts a global skill or agent reads	`.claude/rules/<area>.md`

```
paths: ["**/*public.route.ts"]
```

03

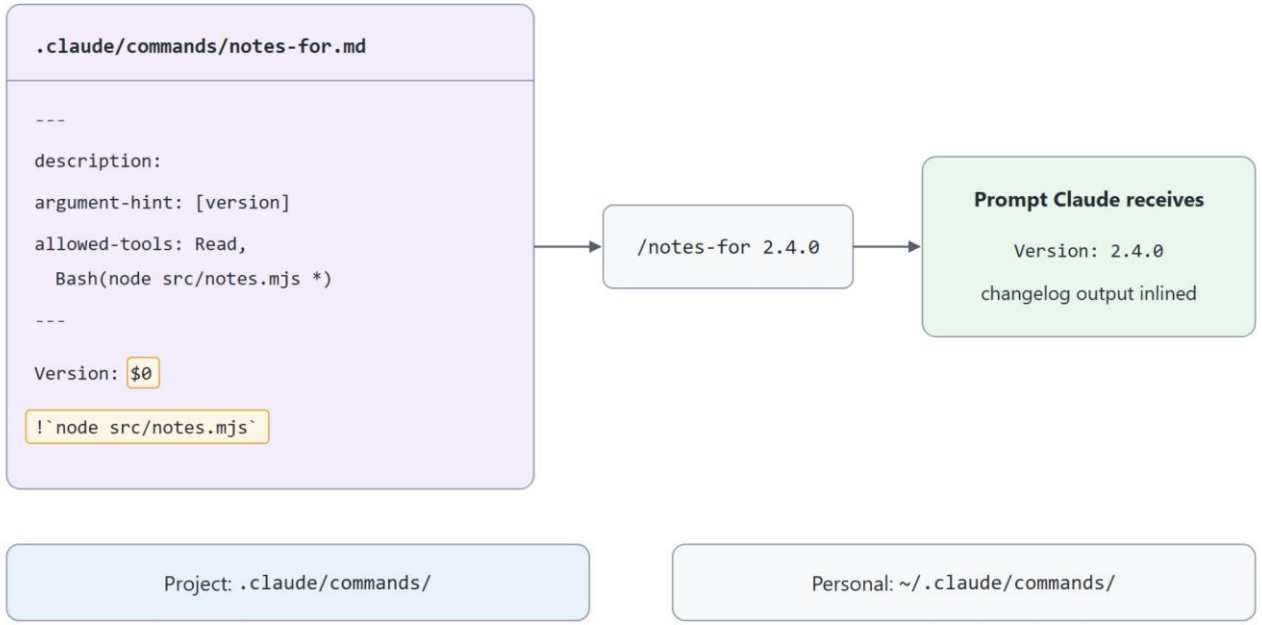
Build Custom Slash Commands

Reusable workflows behind a slash

Custom slash commands

A Markdown file becomes a named workflow

- The file name is the command: `release-status.md` becomes **`/release-status`**
- **`$ARGUMENTS`** takes everything typed after the name; `$0` is the first argument
- `!`command`` runs before Claude reads the file and inlines its output
- **`allowed-tools`** pre-approves calls for one turn; it blocks nothing
- Project scope `.claude/commands/`, personal scope `~/.claude/commands/`



Commands for engineering workflows

Tests, reviews, migrations and releases on demand

- `/run-tests`: run the suite and open every failing file
- `/review` and `/security-review`: structured review of the branch
- `/run-migrations`: apply, verify the schema, roll back on failure
- `/create-pr`: write the description from commits and open a draft
- Built-ins such as `/clear`, `/cost`, `/model` ship with every session



04

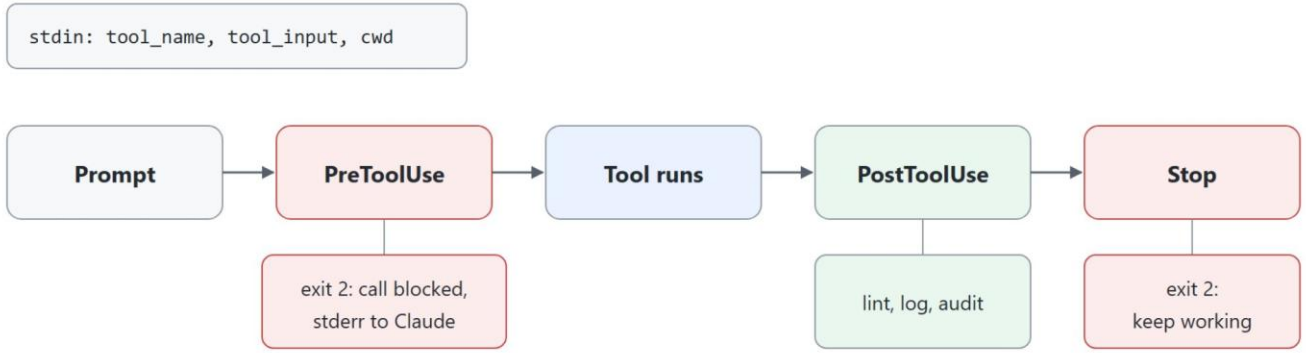
Automate Workflows with Hooks and Mods

Deterministic guardrails and harness extensions

Hook events and exit codes

Shell commands the model cannot argue with

- Run in your OS shell, outside the model: **deterministic**
- Wired in `settings.json`: event, **`matcher`** on the tool name, command
- Payload arrives on **stdin** as JSON: `tool_name`, `tool_input`, `cwd`
- **Exit 2** blocks a `PreToolUse` call and feeds stderr back to Claude
- `PostToolUse` cannot undo a write; it reports the problem for the next turn

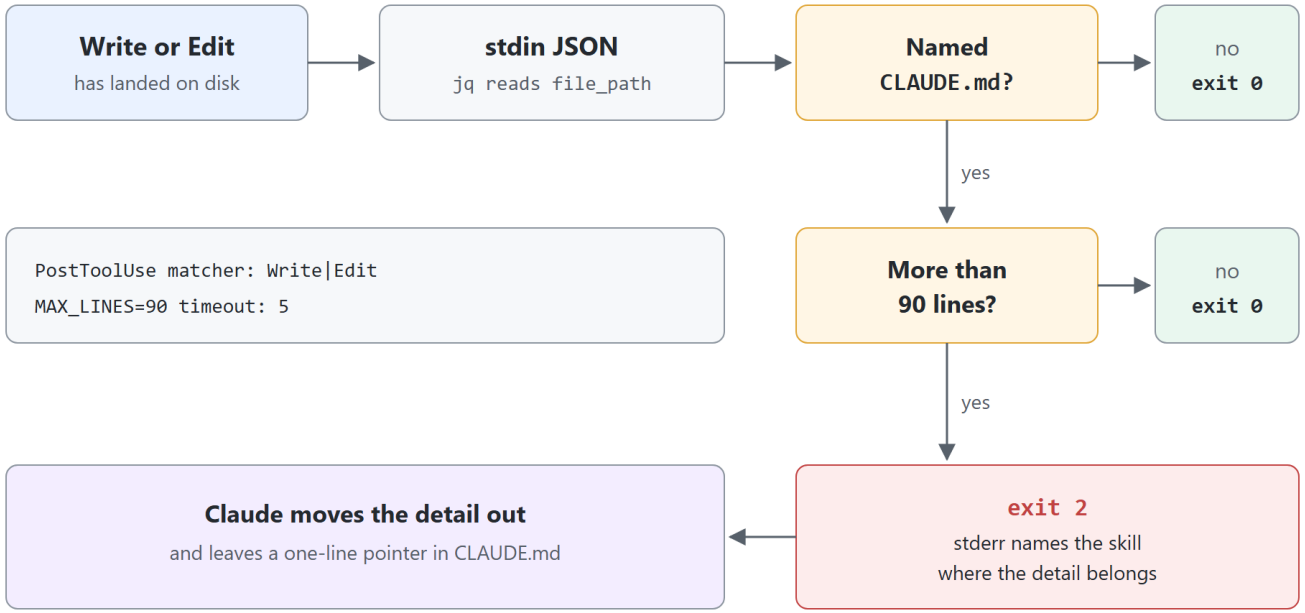


`exit 0`	allow, stdout may carry JSON
`exit 2`	block, stderr is the reason
`other`	non-blocking error

A hook that keeps CLAUDE.md short

One script, one rule, one exit code

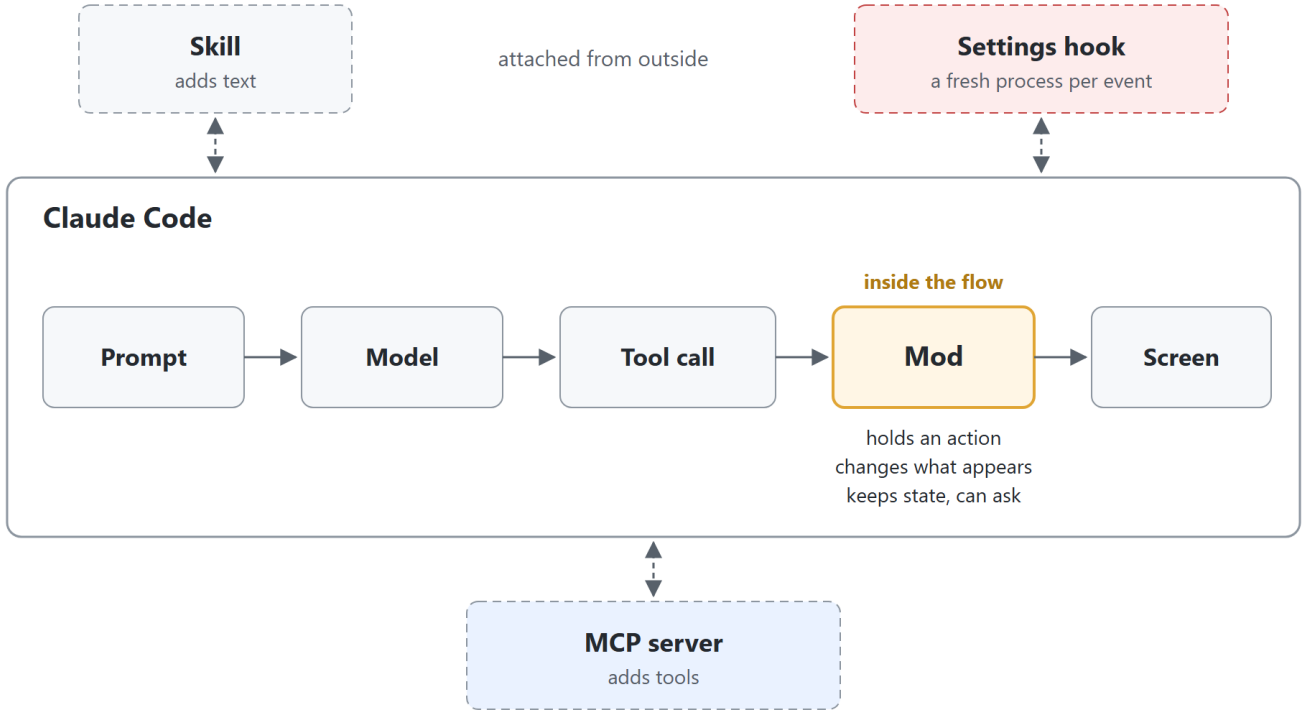
- `claude-md-guard.sh` runs on `PostToolUse` for `Write|Edit`
- `jq` reads the file path from the JSON on **stdin**
- Any other file: **exit 0**, silent
- `CLAUDE.md` over 90 lines: **exit 2**, stderr names the owning skill
- Starts fresh every time: **no memory, no screen**



Hook outside, mod inside

Where the code runs decides what it can do

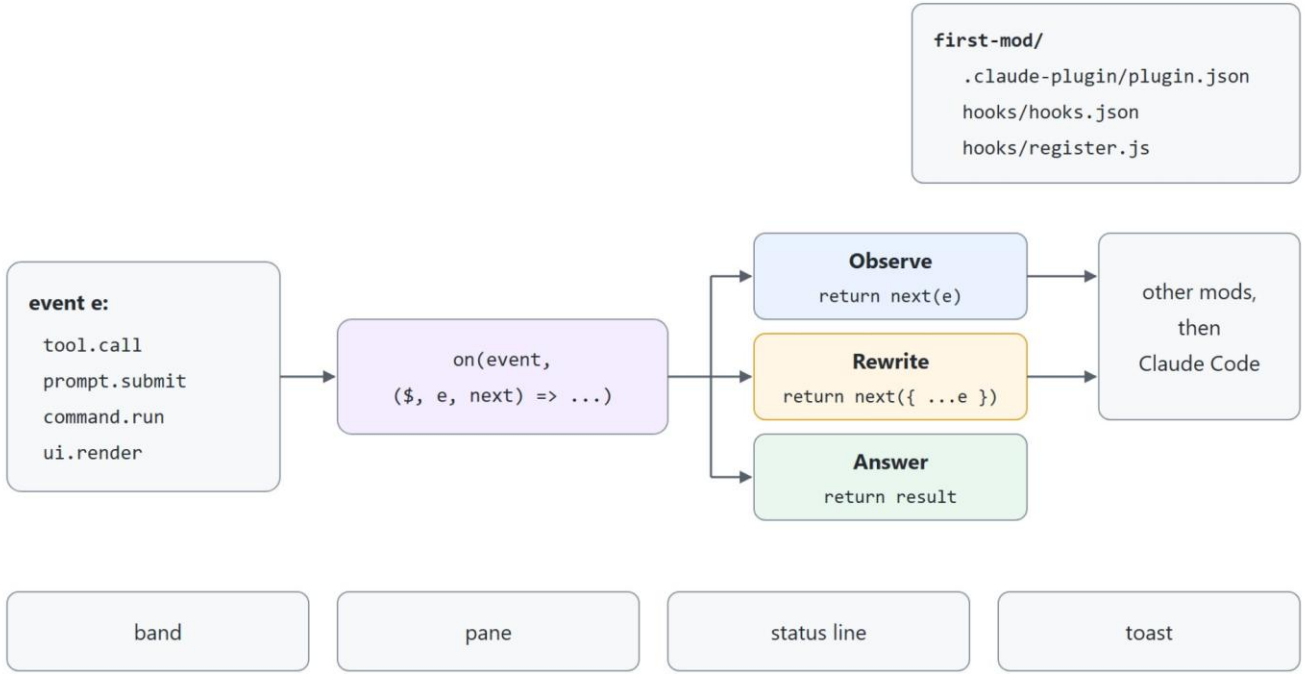
- **Settings hook**: a separate process per event, JSON in, exit code out
- **Mod**: functions inside Claude Code, loaded once with state
- A mod can **hold an action** and **draw**: band, pane, question
- A hook ships with the repo in `.claude/settings.json`
- A mod ships through a **marketplace**: install once per machine



Mods

Functions that run inside Claude Code

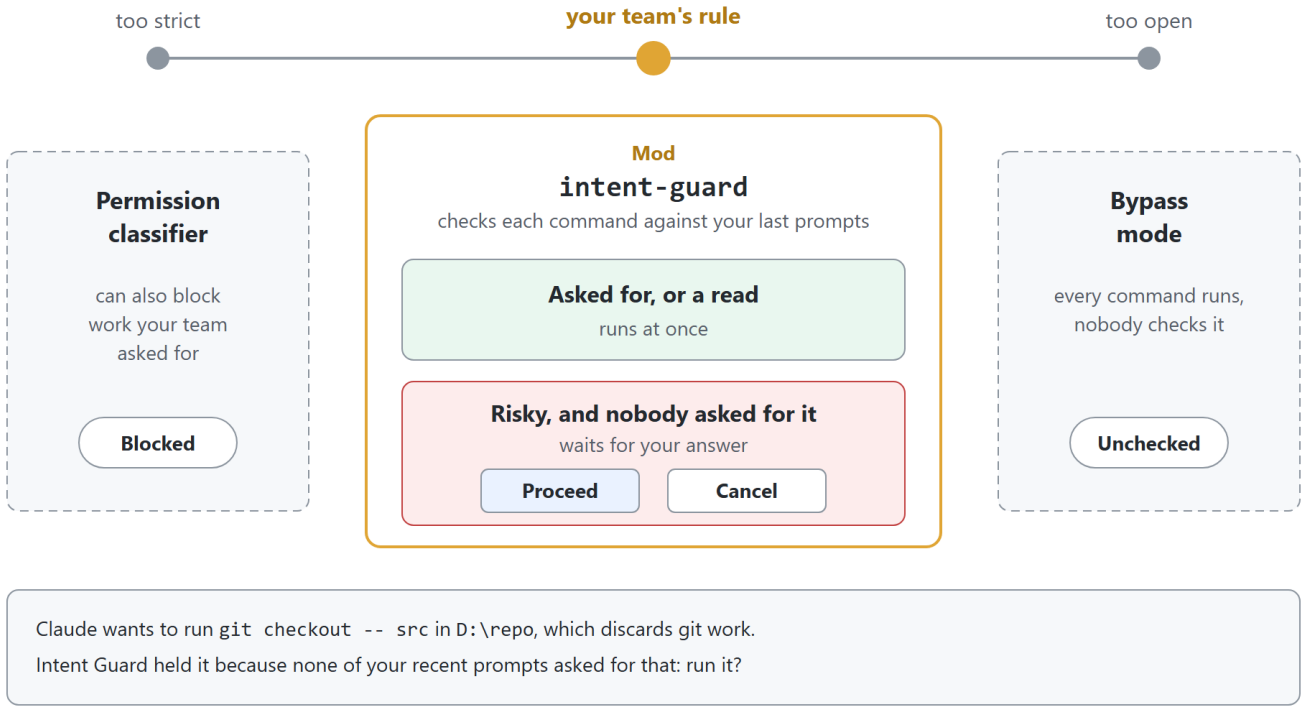
- A plugin whose **`hooks/hooks.json`** names a module under `modules`
- **`register(on)`** subscribes to events such as `tool.call` and `ui.render`
- Each hook can **observe**, **rewrite** or **answer** an event
- Draws bands, panes, status lines and toasts in the terminal and Desktop
- Validate with **`claude plugin validate`**, test with `claude plugin test`



A mod that holds unrequested commands

intent-guard applies your team's rule

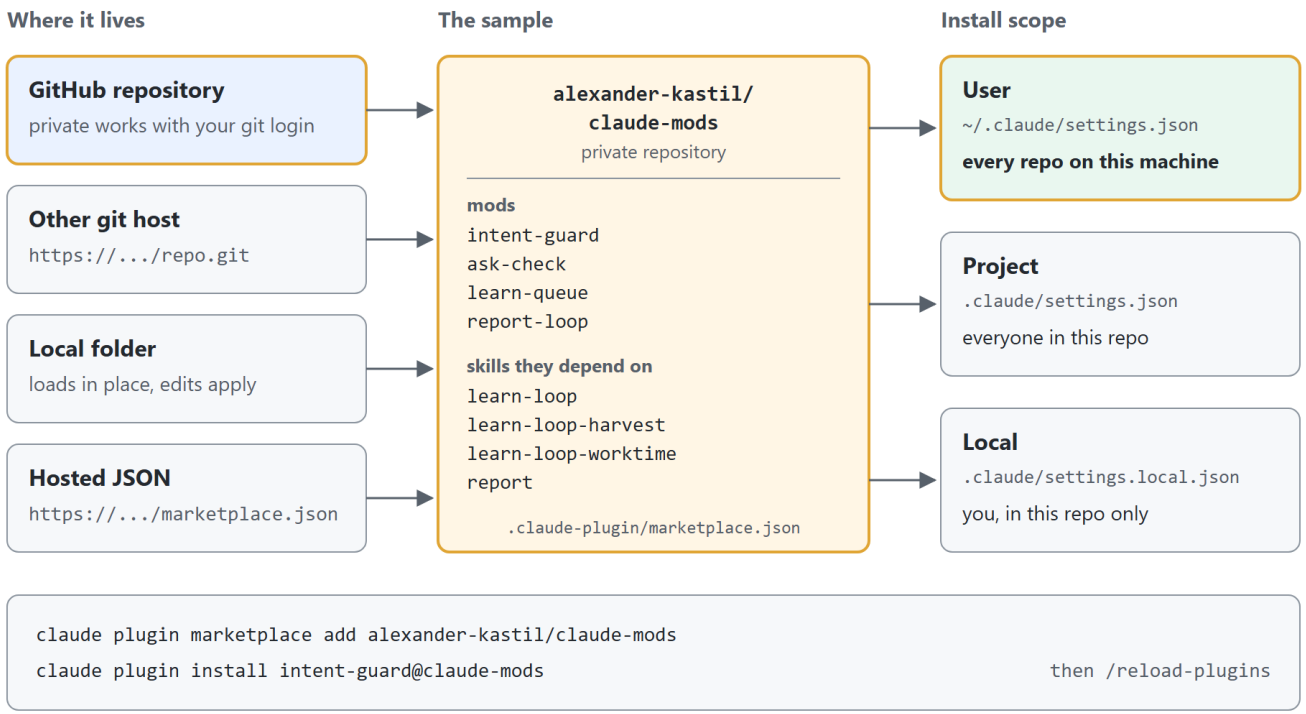
- Sits between the **permission classifier** and **bypass mode**
- Remembers your **last ten prompts** across the session
- Asked for, or a read: **runs at once**
- Risky and unasked: waits for **Proceed** or **Cancel**
- The question names **who**, **what**, **where** and **why**



Install mods from a marketplace

Once per machine, in every repository

- A marketplace is a repo or folder with `marketplace.json`
- **Private** works: Claude Code clones with your git login
- **User scope** is the default: every repo on this machine
- Installing `learn-queue` also installs `learn-loop`
- `/reload-plugins` loads a shell install now



05

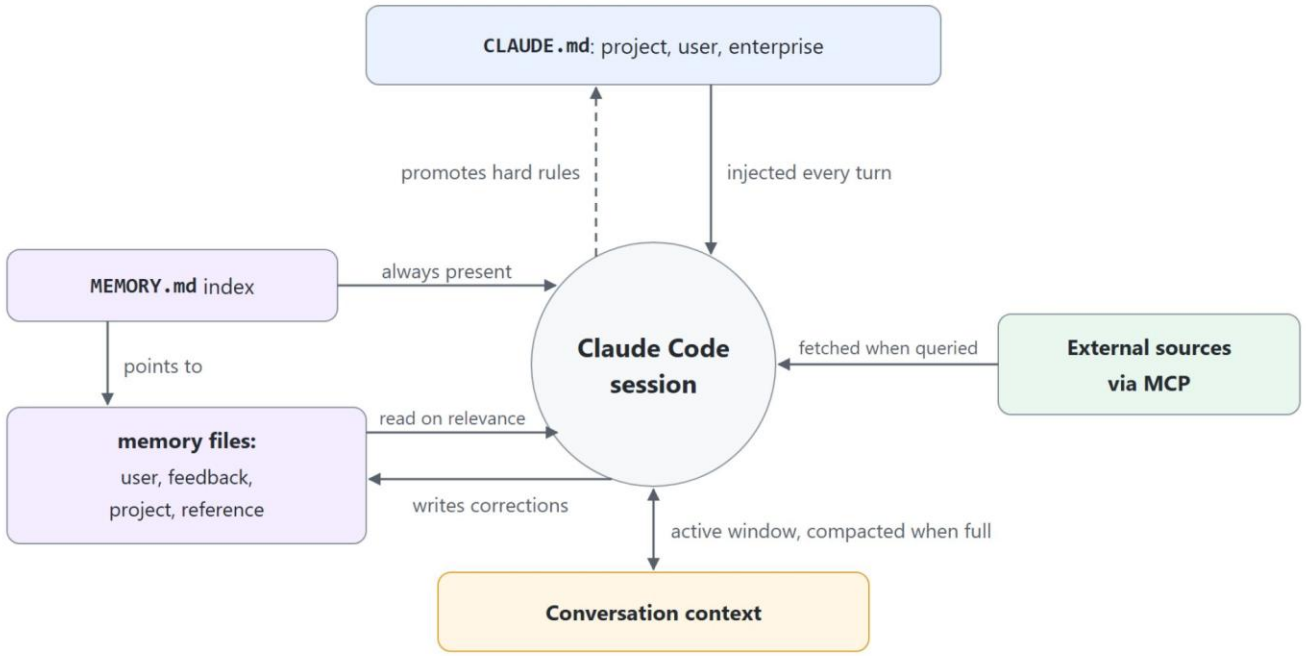
Persist Corrections in Memory Files

Corrections that persist across sessions

Four kinds of memory

When each one loads and who pays for it

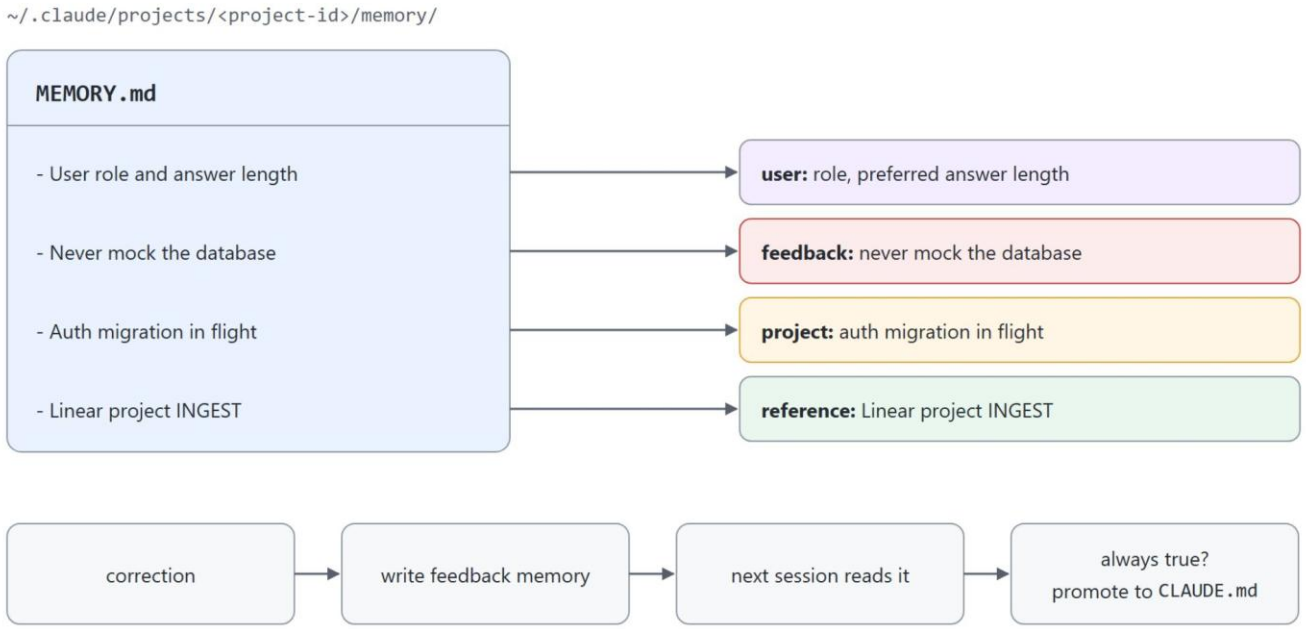
- `CLAUDE.md`: injected every turn, so only always-true rules
- **Memory files**: read when their index line looks relevant
- **External memory** via MCP: fetched only when queried
- **Conversation context**: fastest, re-sent every turn, compacted when full



The MEMORY.md index

One line per memory, bodies read on demand

- Stored in `~/.claude/projects/<project-id>/memory/`
- `MEMORY.md` is always loaded; each body is read only when relevant
- Four types: **user**, **feedback**, **project**, **reference**
- Write a **feedback** memory right after a correction
- A rule that is always true gets **promoted** into `CLAUDE.md`



Takeaways

Which harness file holds what

- Always true: `CLAUDE.md`; true for a file pattern: `.claude/rules/`
- A repeated workflow: **slash command**; a capability with scripts: **skill**
- Live systems and docs: **MCP server**; delegated reading: **subagent**
- Must be enforced: **hook**; must change the harness itself: **mod**
- A correction for tomorrow: **memory file**

